Horário escolar

Uma escola tem um conjunto de turmas (T), de disciplinas (D), de professores (P), com a respetiva disponibilidade, e de tipos de sala (S), normais e especiais, em número limitado. O horário semanal tem 5 dias, cada um com 5 tempos. Cada disciplina d tem um professor $p(d)$, uma carga semanal $c(d)$, um indicador de duplo período e um tipo de sala $s(d)$. Cada tipo de sala s tem Q_s salas disponíveis em simultâneo, e cada professor tem alguns tempos indisponíveis. Todos estes valores são lidos dos ficheiros CSV. Pretende-se gerar automaticamente o horário semanal de cada turma.

Regras

- **R1.** Uma turma não pode ter duas aulas em simultâneo.
- **R2.** Cada disciplina cumpre exatamente a carga semanal exigida para cada turma.
- **R3.** No máximo, só pode haver uma aula da mesma disciplina por dia e por turma.
- **R4.** As disciplinas de duplo período têm de ser dadas em blocos de dois tempos consecutivos, no mesmo dia, e cada bloco conta como uma só aula.
- **R5.** Um professor não pode dar duas aulas em simultâneo, mesmo que sejam a turmas ou disciplinas diferentes.
- **R6.** Um professor só pode dar aulas nos tempos em que está disponível.
- **R7.** Cada aula ocupa uma sala. Disciplinas com `sala_especial` só podem usar salas desse tipo; as restantes usam salas `normal`. Em nenhum tempo o número de aulas a decorrer num tipo de sala pode exceder a `quantidade` desse tipo definida em `salas.csv`.

Objetivo

O número total de "buracos" no horário de cada professor deve ser minimizado. Um buraco é um tempo livre, no meio do dia, entre a primeira e a última aula desse professor nesse dia.

Construção incremental

Quando os recursos mudam ligeiramente, o horário deve ser reajustado sem recomeçar do zero e alterando o menor número possível de aulas.

Análise do problema

Este é um problema de alocação. Pretende-se alocar aulas (disciplinas de cada turma) aos tempos da semana, respeitando a disponibilidade dos professores e as salas existentes.

Existem T turmas, que podemos identificar por um índice $t \in [0..T-1]$, e D disciplinas, identificadas por $d \in [0..D-1]$. Cada tempo da semana é identificado por um par $(i, h) \in [0..4] \times [0..4]$, em que i é o dia e h o tempo desse dia.

Vamos usar uma família $x_{t,d,i,h}$ de variáveis binárias, com a seguinte semântica

$$x_{t,d,i,h} == 1 \quad \text{se e só se} \quad \text{a turma } t \text{ tiver a disciplina } d \text{ no dia } i, \text{ ao tempo } h.$$

Estas $T \times D \times 5 \times 5$ variáveis são convenientemente representadas numa matriz X instanciável com valores $\{0, 1\}^{T \times D \times 5 \times 5}$, a que se costuma chamar *matriz de alocação*.

O professor e o tipo de sala de cada disciplina são dados do problema, e não variáveis de decisão. Por isso não fazem parte do índice de x e só aparecem nas restrições, através de $p(d)$ e $s(d)$.

Destaca-se ainda o seguinte:

Restrições

1. Uma turma não tem duas aulas em simultâneo.
2. Cada turma tem exatamente $c(d)$ tempos semanais de cada disciplina d.
3. Em cada dia, uma turma tem no máximo uma aula de cada disciplina sem duplo período, e no máximo um bloco de dois tempos de cada disciplina com duplo período.
4. As disciplinas de duplo período só podem ser dadas em blocos de dois tempos consecutivos, no mesmo dia.
5. Em cada tempo, um professor tem no máximo uma aula, somando todas as suas disciplinas e turmas.
6. Um professor não dá aulas nos tempos em que está indisponível.
7. Em cada tempo, o número de aulas num tipo de sala s não excede a quantidade de salas Q_s.

Implementação

Começamos por importar o CP-SAT do OR-TOOLS e criar o modelo 'modelo'.

Escolhemos o CP-SAT do OR-Tools porque o problema é naturalmente um problema de satisfação de restrições sobre variáveis booleanas, com um objetivo linear a minimizar: todas as regras se escrevem como somas de variáveis binárias, e o CP-SAT permite ainda sugerir soluções iniciais (`AddHint`), o que é útil na construção incremental. Os

dados são lidos com `pandas`, que lê diretamente os CSV para tabelas e trata as colunas vazias (como `sala_especial`) sem código extra.

Depois obtemos os conjuntos T (turmas) e D (disciplinas) a partir dos dados lidos dos ficheiros CSV, e definimos a estrutura da semana: 5 dias com 5 tempos cada.

Em seguida, declaramos a matriz de alocação X como um dicionário de variáveis booleanas $x_{t,d,i,h}$.

Depois de resolver, o horário é guardado como uma **lista de tuplos** (t, d, i, h), um por cada aula marcada. É neste formato que é passado ao validador, à contagem de buracos e à construção incremental.

In [ ]:
```
import marimo as mo
import time
import copy
```

In [ ]:
```
pasta_dropdown = mo.ui.dropdown(
    options=["dados", "dados_v2", "dados_teste"],
    value="dados",
    label="Conjunto de dados",
)
pasta_dropdown
```

Conjunto de dados dados

In [ ]:
```
import pandas as pd
from pathlib import Path

def ler_dados(pasta):
    pasta = Path(pasta)
    ficheiro_pref = pasta / "preferencias.csv"
    return {
        "disciplinas": pd.read_csv(pasta / "disciplinas.csv",
encoding="utf-8").fillna({"sala_especial": ""}),
        "disponibilidade": pd.read_csv(pasta /
"disponibilidade_excecoes.csv"),
        "salas": pd.read_csv(pasta / "salas.csv"),
        "turmas": pd.read_csv(pasta / "turmas.csv"),
        "preferencias": (pd.read_csv(ficheiro_pref) if
ficheiro_pref.exists()
                         else pd.DataFrame(columns=["professor", "dia",
"periodo", "penalizacao"])),
    }

pasta = pasta_dropdown.value
dados = ler_dados(pasta)
mo.vstack([dados["disciplinas"], dados["disponibilidade"], dados["salas"],
dados["turmas"]])
```

Search...

disciplina str	professor str	carga_semanal int64	duplo_periodo str	sala_especial str
Matemática	Prof. Ana	4	nao	
Português	Prof. Bruno	4	nao	
Ciências	Prof. Carla	3	nao	Laboratório
História	Prof. Diana	2	nao	
Inglês	Prof. Diana	2	nao	
Educação Física	Prof. Eduardo	2	sim	Ginásio

6 rows, 5 columns *No selection*

Search...

professor str unique: 1	dia str unique: 5	periodo int64 
Prof. Eduardo	Seg	1
Prof. Eduardo	Seg	2
Prof. Eduardo	Seg	3
Prof. Eduardo	Ter	1
Prof. Eduardo	Ter	2
Prof. Eduardo	Ter	3
Prof. Eduardo	Qua	1
Prof. Eduardo	Qua	2
Prof. Eduardo	Qua	3
Prof. Eduardo	Qui	1

15 rows, 3 columns *No selection*

Search...

sala str	tipo str	quantidade int64
Sala Normal	normal	6
Laboratório	especial	1
Ginásio	especial	1

3 rows, 3 columns *No selection*

Search...

turma str
7°A
7°B

2 rows, 1 column *No selection*

In [ ]:

```python
def lista_de_dados(dados):
    disc = dados["disciplinas"]
    salas = dados["salas"]
    sala_normal = salas.loc[salas["tipo"] == "normal", "sala"].iloc[0]

    return {
        "T": dados["turmas"]["turma"].tolist(),
        "D": {
            r.disciplina: {
                "professor": r.professor,
                "carga": int(r.carga_semanal),
                "dupla": r.duplo_periodo == "sim",
                "sala": r.sala_especial or sala_normal,
            }
            for r in disc.itertuples()
        },
        "I": ["Seg", "Ter", "Qua", "Qui", "Sex"],
        "H": range(1, 6),
        "salas": dict(zip(salas["sala"], salas["quantidade"])),
```

```
        "indisponivel": set(zip(dados["disponibilidade"]["professor"],
dados["disponibilidade"]["dia"], dados["disponibilidade"]["periodo"])),
        "preferencias": {(r.professor, r.dia, r.periodo):
int(r.penalizacao)
                         for r in dados["preferencias"].itertuples()},
    }
```

In [ ]:
```
from ortools.sat.python import cp_model
```

Passamos agora à modelação das restrições.

A restrição:

1. Uma turma não tem duas aulas em simultâneo

pode expressar-se da seguinte forma:

$$\forall_{t<T} \cdot \forall_{i<I} \cdot \forall_{h<H} \cdot \quad \sum_{d<D} x_{t,d,i,h} \leq 1$$

In [ ]:
```
def restricao_sem_aulas_simultaneo(modelo, x, lista):
    for t in lista["T"]:
        for i in lista["I"]:
            for h in lista["H"]:
                modelo.Add(sum(x[(t, d, i, h)] for d in lista["D"]) <= 1)
```

2. Cada turma tem exatamente $c(d)$ tempos semanais de cada disciplina d

pode expressar-se da seguinte forma:

$$\forall_{t<T} \cdot \forall_{d<D} \cdot \quad \sum_{i<I} \sum_{h<H} x_{t,d,i,h} = c(d)$$

In [ ]:
```
def restricao_carga_semanal(modelo, x, lista):
    for t in lista["T"]:
        for d, info in lista["D"].items():
            modelo.Add(sum(x[(t, d, i, h)] for i in lista["I"] for h in
lista["H"]) == info["carga"])
```

3. Em cada dia, uma turma tem no máximo uma aula de cada disciplina sem duplo período, e no máximo um bloco de dois tempos consecutivos de cada disciplina com duplo período. Divide-se em duas expressões, as com duplo período e as que não têm.

As sem duplo período pode expressar-se da seguinte forma:

$$\forall_{i<I} \cdot \forall_{t<T} \cdot \forall_{d<D,\ duplo(d)=\text{não}} \quad \sum_{h<H} x_{t,d,i,h} \leq 1$$

As com duplo período pode expressar-se da seguinte forma (com $b_{t,d,i}$ binária):

$$\forall_{i<I} \cdot \forall_{t<T} \cdot \forall_{d<D,\ duplo(d)=\text{sim}} \quad \sum_{h<H} x_{t,d,i,h} = 2\, b_{t,d,i}$$

$$\forall_{i<I} \cdot \forall_{t<T} \cdot \forall_{d<D,\ duplo(d)=\text{sim}} \cdot \forall_{h,k<H,\ k>h+1} \quad x_{t,d,i,h} + x_{t,d,i,k} \leq 1$$

In [ ]:
```
def restricao_sem_duplo_periodo(modelo, x, lista):
    for t in lista["T"]:
        for d, info in lista["D"].items():
            if info["dupla"]:
                continue
            for i in lista["I"]:
                modelo.Add(sum(x[(t, d, i, h)] for h in lista["H"]) <= 1)
```

In [ ]:
```
def restricao_com_duplo_periodo(modelo, x, lista):
    for t in lista["T"]:
        for d, info in lista["D"].items():
            if not info["dupla"]:
                continue
            for i in lista["I"]:
                b = modelo.NewBoolVar(f"bloco_{t}_{d}_{i}")
                modelo.Add(sum(x[t, d, i, h] for h in lista["H"]) == 2 * b)
                for h1 in lista["H"]:
                    for h2 in lista["H"]:
                        if h2 > h1 + 1:
                            modelo.Add(x[(t, d, i, h1)] + x[(t, d, i, h2)]
<= 1)
```

4. Em cada tempo, um professor tem no máximo uma aula, somando todas as suas disciplinas e turmas

pode expressar-se da seguinte forma:

$$\forall_{p<P} \cdot \forall_{i<I} \cdot \forall_{h<H} \cdot \quad \sum_{t<T} \sum_{d<D,\ p(d)=p} x_{t,d,i,h} \leq 1$$

In [ ]:
```
def restricao_professor_maximo_uma_aula_por_tempo(modelo, x, lista):
    professores = {info["professor"] for info in lista["D"].values()}
    for q in professores:
        discs = [d for d, info in lista["D"].items() if info["professor"]
== q]
        for i in lista["I"]:
            for h in lista["H"]:
                modelo.Add(sum(x[t, d, i, h] for t in lista["T"] for d in
discs) <= 1)
```

5. Um professor não dá aulas nos tempos em que está indisponível

pode expressar-se da seguinte forma:

$$\forall_{t<T} \cdot \forall_{d<D} \cdot \forall_{i<I} \cdot \forall_{h<H,\ (p(d),i,h)\in Ind} \quad x_{t,d,i,h} = 0$$

In [ ]:
```
def restricao_professor_indisponivel(modelo, x, lista):
    for t in lista["T"]:
        for d, info in lista["D"].items():
            for i in lista["I"]:
                for h in lista["H"]:
                    if (info["professor"], i, h) in lista["indisponivel"]:
                        modelo.Add(x[t, d, i, h] == 0)
```

6. Em cada tempo, o número de aulas num tipo de sala s não excede a quantidade de salas Q_s

pode expressar-se da seguinte forma:

$$\forall_{s<S} \cdot \forall_{i<I} \cdot \forall_{h<H} \cdot \quad \sum_{t<T} \sum_{d<D,\ s(d)=s} x_{t,d,i,h} \leq Q_s$$

In [ ]:
```
def restricao_salas(modelo, x, lista):
    for s, q in lista["salas"].items():
        discs = [d for d, info in lista["D"].items() if info["sala"] == s]
        for i in lista["I"]:
            for h in lista["H"]:
                modelo.Add(sum(x[(t, d, i, h)] for t in lista["T"] for d in
discs) <= q)
```

Validação do Horário

Quando o solver devolve um horário, este cumpre sempre o modelo que lhe demos, mas isso não garante que o modelo esteja bem escrito: uma restrição esquecida ou mal formulada passa despercebida. Por isso escrevemos um **validador independente**, `verificar(pasta, horario)`, que recebe o nome da pasta dos dados e um horário já gerado (lista de tuplos (t, d, i, h)) e confirma, regra a regra, que o horário cumpre o enunciado.

O validador não usa o modelo CP-SAT, as suas variáveis nem `lista_de_dados`: relê os dados dos ficheiros CSV e recalcula tudo a partir do próprio horário. Devolve a lista de violações encontradas, e uma lista vazia significa que o horário é válido.

Testes do validador

Um validador que nunca falha não prova nada. Por isso, para cada requisito, partimos de um horário válido, introduzimos de propósito uma violação desse requisito e confirmamos que `verificar` a deteta. Confirmamos também que o horário gerado pelo solver não tem nenhuma violação.

Dados diferentes

Para mostrar que nada está escrito diretamente no código, repetimos o fluxo completo com um conjunto de dados diferente, em `dados_teste/`, com uma turma e uma disciplina a mais e outra exceção de disponibilidade. Só muda o nome da pasta lida.

In [ ]:

```
from collections import Counter, defaultdict # claude ajudou a corrigir

def verificar(pasta, horario):
    pasta = Path(pasta)
    disc = pd.read_csv(pasta / "disciplinas.csv", encoding="utf-
8").fillna({"sala_especial": ""})
    salas = pd.read_csv(pasta / "salas.csv", encoding="utf-8")
    exc = pd.read_csv(pasta / "disponibilidade_excecoes.csv",
encoding="utf-8")
    turmas = pd.read_csv(pasta / "turmas.csv", encoding="utf-8")
["turma"].tolist()

    sala_normal = salas.loc[salas["tipo"] == "normal", "sala"].iloc[0]
    info = {r.disciplina: r for r in disc.itertuples()}
    indisp = set(zip(exc["professor"], exc["dia"], exc["periodo"]))
    capacidade = dict(zip(salas["sala"], salas["quantidade"]))

    violacoes = []

    # R1: turma sem duas aulas em simultâneo
    c = Counter((t, i, h) for t, d, i, h in horario)
    for (t, i, h), n in c.items():
        if n > 1:
            violacoes.append(("R1", f"{t} tem {n} aulas em {i} tempo {h}"))

    # R2: carga semanal exata
    aulas = Counter((t, d) for t, d, i, h in horario)
    for t in turmas:
        for d, linha in info.items():
            n = aulas[t, d]
            if n != linha.carga_semanal:
                violacoes.append(("R2", f"{t}, {d}: {n} tempos em vez de
{linha.carga_semanal}"))

    # R3: disciplinas sem duplo período
    por_dia = Counter((t, d, i) for t, d, i, h in horario)
    for (t, d, i), n in por_dia.items():
        if info[d].duplo_periodo != "sim" and n > 1:
            violacoes.append(("R3", f"{t}, {d}: {n} aulas em {i}"))

    # R4: disciplinas de duplo período
    tempos = defaultdict(list)
    for t, d, i, h in horario:
        if info[d].duplo_periodo == "sim":
            tempos[t, d, i].append(h)

    for (t, d, i), hs in tempos.items():
        hs = sorted(hs)
        if not (len(hs) == 2 and hs[1] == hs[0] + 1):
            violacoes.append(("R4", f"{t}, {d}, {i}: tempos {hs}"))
```

```
    # R5: professor sem aulas simultâneas
    por_prof = Counter((info[d].professor, i, h) for t, d, i, h in horario)
    for (prof, i, h), n in por_prof.items():
        if n > 1:
            violacoes.append(("R5", f"{prof} tem {n} aulas em {i} tempo
{h}"))

    # R6: nenhuma aula do professor quando está indisponível
    for t, d, i, h in horario:
        prof = info[d].professor
        if (prof, i, h) in indisp:
            violacoes.append(("R6", f"{prof} indisponível em {i} tempo {h}
({t}, {d})"))

    # R7: o número de aulas num tipo de sala não excede a quantidade de
salas
    def sala_de(d):
        return info[d].sala_especial or sala_normal
    por_sala = Counter((sala_de(d), i, h) for t, d, i, h in horario)
    for (s, i, h), n in por_sala.items():
        if n > capacidade[s]:
            violacoes.append(("R7", f"{s}: {n} aulas em {i} tempo {h}
(máximo {capacidade[s]})"))

    return violacoes
```

In [ ]:
```
verificar(pasta, horario)
```

In [ ]:
```
base = list(horario)

def violou(pasta, h, regra):
    return any(r == regra for r, _ in verificar(pasta, h))

def test_valido():
    assert verificar(pasta, base) == []

def test_R1():
    mal = base + [("7ºA", "Matemática", "Seg", 1), ("7ºA", "Português",
"Seg", 1)]
    assert violou(pasta, mal, "R1")

def test_R2():
    mal = [a for a in base if not (a[0] == "7ºA" and a[1] == "Matemática")]
    assert violou(pasta, mal, "R2")

def test_R3():
    mal = base + [("7ºA", "Matemática", "Seg", 1), ("7ºA", "Matemática",
"Seg", 2)]
    assert violou(pasta, mal, "R3")

def test_R4():
    # parte um bloco duplo: troca o 2.º tempo por um não consecutivo
    ef = sorted(a for a in base if a[0] == "7ºA" and a[1] == "Educação
Física")
```

```
    t, d, i, h = ef[1]
    mal = [a for a in base if a != ef[1]] + [(t, d, i, 5 if h != 5 else 1)]
    assert violou(pasta, mal, "R4")

def test_R5():
    mal = base + [("7ºA", "Matemática", "Seg", 1), ("7ºB", "Matemática",
"Seg", 1)]
    assert violou(pasta, mal, "R5")

def test_R6():
    mal = base + [("7ºA", "Educação Física", "Seg", 1)]
    assert violou(pasta, mal, "R6")

def test_R7():
    mal = base + [("7ºA", "Ciências", "Seg", 1), ("7ºB", "Ciências", "Seg",
1)]
    assert violou(pasta, mal, "R7")

def test_contar_buracos():
    # horário feito à mão: o professor de Matemática tem aulas aos tempos 1
e 3 de segunda
    feito_a_mao = [("7ºA", "Matemática", "Seg", 1), ("7ºA", "Matemática",
"Seg", 3)]
    assert contar_buracos(pasta, feito_a_mao) == 1

_testes = [test_valido, test_R1, test_R2, test_R3, test_R4, test_R5,
test_R6, test_R7, test_contar_buracos]
for _f in _testes:
    _f()
print(f"{len(_testes)} testes passaram")
```

```
9 testes passaram
```

Objetivo: minimizar os buracos dos professores

Um **buraco** é um tempo livre de um professor, no meio do dia, com pelo menos uma aula antes e uma aula depois. Os tempos livres no início ou no fim do dia não contam.

Para os contar dentro do modelo, introduzimos três famílias de variáveis binárias, para cada professor p, dia i e tempo h:

- $a_{p,i,h}$ vale 1 se o professor p tem aula no tempo h no dia i;
- $m_{p,i,h}$ vale 1 se o professor p tem uma aula num tempo anterior a h, nesse dia;
- $n_{p,i,h}$ vale 1 se o professor p tem uma aula num tempo posterior a h, nesse dia.

O valor de a obtém-se somando as aulas de todas as disciplinas do professor, em todas as turmas:

$$a_{p,i,h} = \sum_{t<T} \sum_{d<D,\ p(d)=p} x_{t,d,i,h}$$

As variáveis m e n ficam limitadas inferiormente pelos tempos em causa:

$$\forall_{k<h} \cdot m_{p,i,h} \geq a_{p,i,k} \qquad \forall_{k>h} \cdot n_{p,i,h} \geq a_{p,i,k}$$

Basta o limite inferior: se houver aula antes, m é forçada a 1, e o solver nunca ganha em pô-la a 1 sem necessidade, porque isso só pode aumentar os buracos.

Introduzimos ainda uma variável binária $b_{p,i,h}$ por tempo, que indica um buraco. Há buraco quando existe aula antes, existe aula depois e não existe aula no próprio tempo:

$$b_{p,i,h} \geq m_{p,i,h} + n_{p,i,h} - a_{p,i,h} - 1$$

Quando as três condições se verificam, o lado direito vale 1 e b é forçada a 1. Nos outros casos vale 0 ou menos, e b pode ser 0. Em particular, um dia sem aulas do professor não conta buracos. Como o primeiro e o último tempo do dia nunca podem ser buracos, só criamos m, n e b para os tempos interiores.

O objetivo é minimizar o número total de buracos:

$$\min \sum_{p<P} \sum_{i<I} \sum_{h<H} b_{p,i,h}$$

Esta função objetivo não altera as restrições: apenas escolhe, entre os horários válidos, os que têm menos buracos. Por isso o horário continua a passar no validador.

Verificação independente. Para confirmar o resultado sem usar o modelo, a função `contar_buracos` percorre o horário já gerado e, para cada professor e dia, calcula o intervalo entre a primeira e a última aula menos o número de aulas. Depois comparamos este valor com `solver.ObjectiveValue()`: se a solução for `OPTIMAL`, têm de ser iguais (somando o custo das preferências, quando existem).

In [ ]:
```
def menos_buracos(modelo, x, lista, extra=0): # claude ajudou a corrigir
    H = list(lista["H"])
    professores = sorted({info["professor"] for info in
lista["D"].values()})
    buracos = []
    penal = []
    for prof in professores:
        discs = [d for d, info in lista["D"].items() if info["professor"]
== prof]
        for i in lista["I"]:
            a = {}
            for h in H:
                a[h] = modelo.NewBoolVar(f"a_{prof}_{i}_{h}")
                modelo.Add(a[h] == sum(x[t, d, i, h] for t in lista["T"]
for d in discs))
                peso = lista["preferencias"].get((prof, i, h), 0)
                if peso:
                    penal.append(peso * a[h])
            for h in H[1:-1]:
                antes = modelo.NewBoolVar(f"antes_{prof}_{i}_{h}")
                depois = modelo.NewBoolVar(f"depois_{prof}_{i}_{h}")
```

```
                for k in H:
                    if k < h:
                        modelo.Add(antes >= a[k])
                    elif k > h:
                        modelo.Add(depois >= a[k])
                b = modelo.NewBoolVar(f"b_{prof}_{i}_{h}")
                modelo.Add(b >= antes + depois - a[h] - 1)
                buracos.append(b)

    modelo.Minimize(sum(buracos) + sum(penal) + extra)
    return buracos
```

In [ ]:
```
def contar_buracos(pasta, horario): # claude ajudou a corrigir
    disc = pd.read_csv(Path(pasta) / "disciplinas.csv", encoding="utf-8")
    prof = dict(zip(disc["disciplina"], disc["professor"]))
    por_dia = defaultdict(set)
    for t, d, i, h in horario:
        por_dia[prof[d], i].add(h)
    return sum((max(hs) - min(hs) + 1) - len(hs) for hs in
por_dia.values())

def custo_preferencias(pasta, horario):
    pasta = Path(pasta)
    ficheiro = pasta / "preferencias.csv"
    if not ficheiro.exists():
        return 0
    pref = pd.read_csv(ficheiro, encoding="utf-8")
    pen = {(r.professor, r.dia, r.periodo): int(r.penalizacao) for r in
pref.itertuples()}
    disc = pd.read_csv(pasta / "disciplinas.csv", encoding="utf-8")
    prof = dict(zip(disc["disciplina"], disc["professor"]))
    return sum(pen.get((prof[d], i, h), 0) for t, d, i, h in horario)
```

In [ ]:
```
def construir_modelo(dados, com_objetivo=True):
    lista = lista_de_dados(dados)
    modelo = cp_model.CpModel()
    x = {(t, d, i, h): modelo.NewBoolVar(f"x_{t}_{d}_{i}_{h}")
         for t in lista["T"] for d in lista["D"] for i in lista["I"] for h
in lista["H"]}

    restricao_sem_aulas_simultaneo(modelo, x, lista)
    restricao_sem_duplo_periodo(modelo, x, lista)
    restricao_com_duplo_periodo(modelo, x, lista)
    restricao_professor_maximo_uma_aula_por_tempo(modelo, x, lista)
    restricao_professor_indisponivel(modelo, x, lista)
    restricao_salas(modelo, x, lista)
    restricao_carga_semanal(modelo, x, lista)

    if com_objetivo:
        menos_buracos(modelo, x, lista)
    return modelo, x
```

In [ ]:
```
def resolver(modelo, x, tempo_max=None): # claude ajudou a corrigir
    solver = cp_model.CpSolver()
    if tempo_max:
```

```
        solver.parameters.max_time_in_seconds = tempo_max
    solver.parameters.num_workers = 1        # resultados reprodutíveis
    solver.parameters.random_seed = 0
    estado = solver.Solve(modelo)
    solver.estado = solver.StatusName(estado)
    if estado not in (cp_model.OPTIMAL, cp_model.FEASIBLE):
        return None
    return solver
```

In [ ]:
```
def extrair_horario(solver, x):
    # Converte a solução numa lista de tuplos (t, d, i, h), uma por aula.
    if solver is None:
        return None
    return [(t, d, i, h) for (t, d, i, h), var in x.items() if
solver.Value(var)]
```

In [ ]:
```
def contar(f, modelo, x, p):
    antes = len(modelo.Proto().constraints)
    f(modelo, x, p)
    print(f.__name__, len(modelo.Proto().constraints) - antes)
```

In [ ]:
```
def modelo_vazio(dados):
    lista = lista_de_dados(dados)
    modelo = cp_model.CpModel()
    x = {(t, d, i, h): modelo.NewBoolVar(f"x_{t}_{d}_{i}_{h}")
         for t in lista["T"] for d in lista["D"] for i in lista["I"] for h
in lista["H"]}
    return modelo, x, lista

_modelo, _x, _lista = modelo_vazio(dados)
contar(restricao_sem_aulas_simultaneo, _modelo, _x, _lista)
contar(restricao_sem_duplo_periodo, _modelo, _x, _lista)
contar(restricao_com_duplo_periodo, _modelo, _x, _lista)
contar(restricao_professor_maximo_uma_aula_por_tempo, _modelo, _x, _lista)
contar(restricao_professor_indisponivel, _modelo, _x, _lista)
contar(restricao_salas, _modelo, _x, _lista)
contar(restricao_carga_semanal, _modelo, _x, _lista)
```

```
restricao_sem_aulas_simultaneo 50
restricao_sem_duplo_periodo 50
restricao_com_duplo_periodo 70
restricao_professor_maximo_uma_aula_por_tempo 125
restricao_professor_indisponivel 30
restricao_salas 75
restricao_carga_semanal 12
```

In [ ]:
```
modelo, x = construir_modelo(dados)
solver = resolver(modelo, x)
assert solver is not None, "Sem solução: modelo infeasible ou tempo
esgotado"

horario = extrair_horario(solver, x)
print(type(horario), len(horario))
```

```
<class 'list'> 34
```

In [ ]:
```
_buracos = contar_buracos(pasta, horario)
_pref = custo_preferencias(pasta, horario)
print("Estado:", solver.estado)
print("Objetivo do solver:", solver.ObjectiveValue())
print("Buracos (independente):", _buracos, "| preferências:", _pref, "|
total:", _buracos + _pref)
```

```
Estado: OPTIMAL
Objetivo do solver: 0.0
Buracos (independente): 0 | preferências: 0 | total: 0
```

In [ ]:
```
_lista = lista_de_dados(dados)

_m1, _x1 = construir_modelo(dados, com_objetivo=False)
_h1 = extrair_horario(resolver(_m1, _x1), _x1)
print("Sem objetivo:", contar_buracos(pasta, _h1), "buracos")

_m2, _x2 = construir_modelo(dados, com_objetivo=True)
_h2 = extrair_horario(resolver(_m2, _x2), _x2)
print("Com objetivo:", contar_buracos(pasta, _h2), "buracos")
print("Horário válido:", verificar(pasta, _h2) == [])

_ocupado = {(t, i, h): d for t, d, i, h in _h2}
for _t in _lista["T"]:
    print(_t)
    for _h in _lista["H"]:
        _linha = [_ocupado.get((_t, _i, _h), "-") for _i in _lista["I"]]
        print(f"{_h}º tempo:", " | ".join(f"{a:<16}" for a in _linha))
    print()
```

```
Sem objetivo: 2 buracos
Com objetivo: 0 buracos
Horário válido: True
7ºA
1º tempo: Português        | -               | -               | Ciências
| Português
2º tempo: Ciências         | -               | -               | Inglês
| Ciências
3º tempo: Matemática       | -               | Inglês          | Português
| Matemática
4º tempo: Educação Física  | Matemática      | História        | História
| -
5º tempo: Educação Física  | Português       | Matemática      | -
| -

7ºB
1º tempo: Ciências         | História        | -               | -
| Ciências
2º tempo: Inglês           | Ciências        | -               | Matemátic
a       | Português
3º tempo: História         | -               | Português       | Inglês
| -
4º tempo: Matemática       | Português       | Educação Física  | Português
| Matemática
5º tempo: -                | Matemática      | Educação Física  | -
| -
```

Construção Incremental

Quando os recursos mudam ligeiramente, resolver o problema do zero tem dois problemas: é mais lento e, sobretudo, o solver escolhe um horário qualquer entre os que cumprem as regras, que costuma ser muito diferente do anterior. Para a escola isso significa mudar aulas que não precisavam de mudar.

A nossa abordagem usa o horário anterior H_0 (lista de aulas) de duas formas:

1. **Ponto de partida.** Sugerimos ao solver os valores de H_0 (`AddHint`), para começar a procura perto de uma solução conhecida.
2. **Custo por mudança.** Acrescentamos ao objetivo um termo que penaliza cada aula de H_0 que deixa de estar em H_1:

$$\min \quad \text{peso} \cdot \sum_{(t,d,i,h) \,\in\, H_0} \left(1 - x^{1}_{t,d,i,h}\right) \;+\; \sum_{p,i,h} b_{p,i,h}$$

O `peso` (100) é muito maior do que o número de buracos que se pode ganhar, por isso a estabilidade tem prioridade sobre os buracos. Isto é uma escolha: aceitamos mais buracos se isso evitar mover uma aula.

O H_1 cumpre as mesmas regras R1–R7, porque usa o mesmo `construir_modelo` com os dados novos. As aulas de H_0 que não existem no novo modelo (por exemplo,

de uma turma que desapareceu) são ignoradas. O número de aulas alteradas é o número de aulas de H_0 que não estão em H_1.

Resultados com `dados` → `dados_v2` (tempo médio de 5 execuções, só da resolução):

	Tempo	Aulas alteradas	Válido
Do zero	0,054 s	25	sim
Incremental	0,038 s	2	sim

O ganho principal é na estabilidade: o horário novo mantém quase tudo e muda 2 aulas, em vez de 25. A diferença de tempo existe mas é pequena, porque a instância é pequena e ambos os métodos resolvem em milissegundos. O benefício de velocidade só deve aparecer em instâncias maiores.

Outros cenários

Para mostrar que a abordagem não depende do cenário de `dados_v2`, aplicámo-la a mais alterações de recursos, todas feitas só sobre os dados, sem mexer no código. Os dados alterados são gravados numa pasta própria (`guardar_dados`), para que o validador os possa reler como nos outros casos.

Alteração	Aulas alteradas (incremental)	Aulas alteradas (do zero)
Disponibilidade (`dados_v2`)	2	25
Turma e disciplina novas	3	27
Professor substituído	0	30
Sala Normal com menos salas	6	33

Em todos os casos os dois horários são válidos, e o incremental muda muito menos aulas. A avaria de uma sala modela-se baixando a `quantidade` do tipo em `salas.csv`, porque o formato dos dados só guarda a quantidade por tipo de sala e não a disponibilidade por tempo.

In [ ]:
```
def construcao_incremental(dados_novos, horario0, peso=100): # claude ajudou a corrigir
    modelo, x = construir_modelo(dados_novos, com_objetivo=False)
    p = lista_de_dados(dados_novos)
    if peso is None:
        # maior do que o número máximo de buracos possível, para a estabilidade ganhar sempre
        n_prof = len({info["professor"] for info in p["D"].values()})
        peso = n_prof * len(p["I"]) * max(len(p["H"]) - 2, 1) + 1
    anteriores = set(horario0)
```

```
    mudancas = sum(1 - x[k] for k in anteriores if k in x)
    menos_buracos(modelo, x, p, extra=peso * mudancas)
    for k, var in x.items():
        modelo.AddHint(var, 1 if k in anteriores else 0)
    return modelo, x
```

In [ ]:
```
def mudancas_entre(horario_antigo, horario_novo):
    # número de aulas do horário antigo que já não estão no novo
    return len(set(horario_antigo) - set(horario_novo))
```

In [ ]:
```
dados0 = ler_dados("dados")
dados1 = ler_dados("dados_v2")
```

In [ ]:
```
modelo0, x0 = construir_modelo(dados0)
solver0 = resolver(modelo0, x0)
horario0 = extrair_horario(solver0, x0)

modelo1, x1 = construcao_incremental(dados1, horario0)
solver1 = resolver(modelo1, x1)
horario1 = extrair_horario(solver1, x1)
```

In [ ]:
```
{
    "violações H0": verificar("dados", horario0),
    "violações H1": verificar("dados_v2", horario1),
    "aulas alteradas": mudancas_entre(horario0, horario1),
}
```

In [ ]:
```
def medir(construir, repeticoes=5):
    tempos = []
    for _ in range(repeticoes):
        modelo, x = construir()
        t0 = time.perf_counter()
        solver = resolver(modelo, x)
        tempos.append(time.perf_counter() - t0)
    return extrair_horario(solver, x), sum(tempos) / len(tempos)

horario1_zero, t_zero = medir(lambda: construir_modelo(dados1))
horario1_inc, t_inc = medir(lambda: construcao_incremental(dados1,
horario0))

print(f"Do zero: {t_zero:.3f} s, {mudancas_entre(horario0, horario1_zero)}
aulas alteradas")
print(f"Incremental: {t_inc:.3f} s, {mudancas_entre(horario0,
horario1_inc)} aulas alteradas")
print("Válidos:", verificar("dados_v2", horario1_zero) == [],
verificar("dados_v2", horario1_inc) == [])
```

```
Do zero: 0.027 s, 30 aulas alteradas
Incremental: 0.018 s, 1 aulas alteradas
Válidos: True True
```

In [ ]:
```
def guardar_dados(dados, pasta):
    pasta = Path(pasta)
    pasta.mkdir(exist_ok=True)
    dados["turmas"].to_csv(pasta / "turmas.csv", index=False)
```

```
    dados["disciplinas"].to_csv(pasta / "disciplinas.csv", index=False,
encoding="utf-8")
    dados["salas"].to_csv(pasta / "salas.csv", index=False)
    dados["disponibilidade"].to_csv(pasta / "disponibilidade_excecoes.csv",
index=False)
    if len(dados["preferencias"]):
        dados["preferencias"].to_csv(pasta / "preferencias.csv",
index=False)
```

In [ ]:
```
dados3 = copy.deepcopy(dados0)      # professor substituído
dados3["disciplinas"].loc[dados3["disciplinas"]["disciplina"] ==
"Matemática", "professor"] = "Prof. Nova"
guardar_dados(dados3, "dados_prof_substituido")

dados4 = copy.deepcopy(dados0)      # sala avariada: menos salas normais
dados4["salas"].loc[dados4["salas"]["sala"] == "Sala Normal", "quantidade"]
= 1
guardar_dados(dados4, "dados_sala_reduzida")

for _nome, _pasta, _dn in [("Professor substituído",
"dados_prof_substituido", dados3),
                           ("Sala Normal reduzida", "dados_sala_reduzida",
dados4)]:
    _mi, _xi = construcao_incremental(_dn, horario0)
    _hi = extrair_horario(resolver(_mi, _xi), _xi)
    _mz, _xz = construir_modelo(_dn)
    _hz = extrair_horario(resolver(_mz, _xz), _xz)
    if _hi is None or _hz is None:
        print(_nome, "| sem solução")
        continue
    print(_nome, "| incremental:", mudancas_entre(horario0, _hi),
          "| do zero:", mudancas_entre(horario0, _hz),
          "| válidos:", verificar(_pasta, _hi) == [], verificar(_pasta,
_hz) == [])
```

```
Professor substituído | incremental: 0 | do zero: 25 | válidos: True True
Sala Normal reduzida | incremental: 7 | do zero: 30 | válidos: True True
```

In [ ]:
```
_dados_t = ler_dados("dados_teste")

_mi, _xi = construcao_incremental(_dados_t, horario0)
_hi = extrair_horario(resolver(_mi, _xi), _xi)

_mz, _xz = construir_modelo(_dados_t)
_hz = extrair_horario(resolver(_mz, _xz), _xz)

print("Incremental:", mudancas_entre(horario0, _hi), "aulas alteradas")
print("Do zero:", mudancas_entre(horario0, _hz), "aulas alteradas")
print("Válidos:", verificar("dados_teste", _hi) == [],
verificar("dados_teste", _hz) == [])
```

```
Incremental: 2 aulas alteradas
Do zero: 28 aulas alteradas
Válidos: True True
```

In [ ]:
```
def grelha_df(dados, horario, turma):
    p = lista_de_dados(dados)
```

```
    ocupado = {(t, i, h): d for t, d, i, h in horario}
    linhas = {}
    for h in p["H"]:
        linhas[f"{h}º tempo"] = {i: ocupado.get((turma, i, h), "-") for i
in p["I"]}
    return pd.DataFrame.from_dict(linhas, orient="index")
```

In [ ]:
```
mo.vstack([
    item
    for _t in lista_de_dados(dados)["T"]
    for item in (mo.md(f"**Turma {_t}**"), grelha_df(dados, horario, _t))
])
```

Turma 7ºA

Search...

Index0 str	Seg str	Ter str	Qua str	Qui str	Sex str
1º tempo	Português	-	-	Ciências	Português
2º tempo	Ciências	-	-	Inglês	Ciências
3º tempo	Matemática	-	Inglês	Português	Matemática
4º tempo	Educação Física	Matemática	História	História	-
5º tempo	Educação Física	Português	Matemática	-	-

5 rows, 5 columns — *No selection*

Turma 7ºB

Search...

Index0 str	Seg str	Ter str	Qua str	Qui str	Sex str
1º tempo	Ciências	História	-	-	Ciências
2º tempo	Inglês	Ciências	-	Matemática	Português
3º tempo	História	-	Português	Inglês	-
4º tempo	Matemática	Português	Educação Física	Português	Matemática
5º tempo	-	Matemática	Educação Física	-	-

5 rows, 5 columns — *No selection*

In [ ]:
```
mo.hstack([
    mo.vstack([mo.md("**7ºA, H0**"), grelha_df(dados0, horario0, "7ºA")]),
    mo.vstack([mo.md("**7ºA, H1**"), grelha_df(dados1, horario1, "7ºA")]),
])
```

7ºA, H0

Search...

Index0 str	Seg str	Ter str	Qua str	Qui str	Sex str
1º tempo	Português	-	-	Ciências	Português
2º tempo	Ciências	-	-	Inglês	Ciências
3º tempo	Matemática	-	Inglês	Português	Matemática
4º tempo	Educação Física	Matemática	História	História	-
5º tempo	Educação Física	Português	Matemática	-	-

5 rows, 5 columns — *No selection*

7ºA, H1

Search...

Index0 str	Seg str	Ter str	Qua str	Qui str	Sex str
1º tempo	Português	-	-	Ciências	Português
2º tempo	Ciências	-	-	Inglês	Ciências
3º tempo	Matemática	-	Inglês	Português	Matemática
4º tempo	Educação Física	Matemática	História	História	-
5º tempo	Educação Física	Português	Matemática	-	-

5 rows, 5 columns — *No selection*

Escala (bónus)

Para estudar o comportamento com mais turmas, geramos conjuntos de dados sintéticos (`gerar_dados_escala`) com 16 disciplinas de carga semanal 1, cada uma

com um professor próprio, e 4 a 24 turmas. Os dados continuam a ser lidos de ficheiros CSV, sem alterar o código do modelo.

Turmas	Variáveis	Tempo de resolução (s)	Resultado
4	1 600	0,22	válido
8	3 200	0,35	válido
12	4 800	0,48	válido
16	6 400	0,79	válido
20	8 000	0,86	válido
24	9 600	1,05	válido

Em todos os casos o horário cumpre as regras (verificado pelo validador independente) e o tempo cresce de forma aproximadamente linear com o número de variáveis, ficando perto de 1 s para 9 600 variáveis.

Limites encontrados. Até às 24 turmas não encontrámos um limite computacional, mas encontrámos um limite estrutural dos dados. Com o formato atual, cada turma tem todas as disciplinas e cada disciplina um único professor, pelo que a carga de um professor cresce com o número de turmas. Com 8 disciplinas de carga 2, um professor dá $2N$ tempos por semana e só tem 25, logo o problema fica impossível a partir de 13 turmas (confirmámos com 16: o solver prova rapidamente que não há solução). Por isso os dados de escala usam carga 1 e mais disciplinas, o que permite até 25 turmas.

A dificuldade de uma instância depende não só do tamanho mas de quão apertada está: com salas, professores ou tempos muito escassos, o solver tem de explorar muito mais, e é aí que o tempo pode crescer depressa.

In [ ]:
```
def gerar_dados_escala(pasta, n_turmas, n_disc=16, carga=1): # claude
ajudou a criar
    pasta = Path(pasta)
    pasta.mkdir(exist_ok=True)
    pd.DataFrame({"turma": [f"T{k}" for k in
range(n_turmas)]}).to_csv(pasta / "turmas.csv", index=False)
    pd.DataFrame({
        "disciplina": [f"Disc{j}" for j in range(n_disc)],
        "professor": [f"Prof{j}" for j in range(n_disc)],
        "carga_semanal": [carga] * n_disc,
        "duplo_periodo": ["nao"] * n_disc,
        "sala_especial": [""] * n_disc,
    }).to_csv(pasta / "disciplinas.csv", index=False)
    pd.DataFrame({"sala": ["Sala Normal"], "tipo": ["normal"],
"quantidade": [n_turmas]}).to_csv(pasta / "salas.csv", index=False)
    pd.DataFrame({"professor": [], "dia": [], "periodo": []}).to_csv(pasta
/ "disponibilidade_excecoes.csv", index=False)
```

In [ ]:

```
resultados = []
for _n in (4, 8, 12, 16, 20, 24):
    _pasta = f"dados_escala_{_n}"
    gerar_dados_escala(_pasta, _n)
    _d = ler_dados(_pasta)
    _modelo, _x = construir_modelo(_d)
    _t0 = time.perf_counter()
    _solver = resolver(_modelo, _x, tempo_max=60)
    _dt = time.perf_counter() - _t0
    resultados.append({
        "turmas": _n,
        "variáveis": len(_x),
        "tempo (s)": round(_dt, 2),
        "resultado": "sem solução" if _solver is None
            else ("válido" if verificar(_pasta, extrair_horario(_solver,
_x)) == [] else "INVÁLIDO"),
    })

pd.DataFrame(resultados)
```

Search...

turmas int64	variáveis int64	tempo (s) float64	resultado str
4	1,600	0.11	válido
8	3,200	0.20	válido
12	4,800	0.28	válido
16	6,400	0.79	válido
20	8,000	0.50	válido
24	9,600	0.58	válido

6 rows, 4 columns

No selection

Preferências dos professores (bónus)

Além dos buracos, os professores podem indicar tempos em que preferem não dar aulas. Estas preferências são lidas de `preferencias.csv` (professor, dia, período e penalização), um ficheiro opcional: se não existir, o modelo comporta-se como antes.

Uma preferência não é uma restrição, é um custo: dar uma aula num tempo penalizado soma a penalização ao objetivo. O objetivo passa a ser

$$\min \sum_{p,i,h} b_{p,i,h} + \sum_{p,i,h} \mathrm{pen}_{p,i,h} \cdot a_{p,i,h}$$

em que $\mathrm{pen}_{p,i,h}$ é a penalização lida do ficheiro (0 se não houver). Uma penalização de 1 vale tanto como um buraco, e valores maiores dão mais peso à preferência. Como é um custo e não uma regra, o validador não a verifica: o horário pode, em princípio, usar um tempo penalizado se compensar noutro critério. O custo é recalculado de forma independente por `custo_preferencias(pasta, horario)`.

Resultado. Com o Prof. Bruno a preferir não dar aulas aos dois primeiros tempos de cada dia (penalização 3), o horário gerado sem considerar as preferências tem um custo de 9 (três aulas nesses tempos), e com elas o custo desce para 0. O número de

buracos mantém-se em 0 e o horário continua válido. Os dados deste exemplo são gravados em `dados_preferencias`, para que o custo seja avaliado a partir dos ficheiros, tal como no validador.

In [ ]:
```
_dp = copy.deepcopy(ler_dados("dados")) # claude ajudou a testar
_dp["preferencias"] = pd.DataFrame({
    "professor": ["Prof. Bruno"] * 10,
    "dia": ["Seg", "Ter", "Qua", "Qui", "Sex"] * 2,
    "periodo": [1] * 5 + [2] * 5,
    "penalizacao": [3] * 10,
})
guardar_dados(_dp, "dados_preferencias")      # o custo é avaliado a partir
desta pasta

_dsem = copy.deepcopy(_dp)
_dsem["preferencias"] = _dsem["preferencias"].iloc[0:0]      # mesmos
dados, sem preferências

_mc, _xc = construir_modelo(_dp)
_h_com = extrair_horario(resolver(_mc, _xc), _xc)
_ms, _xs = construir_modelo(_dsem)
_h_sem = extrair_horario(resolver(_ms, _xs), _xs)

print("Custo das preferências sem elas:",
custo_preferencias("dados_preferencias", _h_sem))
print("Custo das preferências com elas:",
custo_preferencias("dados_preferencias", _h_com))
print("Buracos sem elas:", contar_buracos("dados_preferencias", _h_sem))
print("Buracos com elas:", contar_buracos("dados_preferencias", _h_com))
print("Válido:", verificar("dados_preferencias", _h_com) == [])
```

```
Custo das preferências sem elas: 9
Custo das preferências com elas: 0
Buracos sem elas: 0
Buracos com elas: 0
Válido: True
```

Uso de Ferramenta LLM

Durante a realização do trabalho foi utilizado o Claude como ferramenta de apoio à implementação, depuração e revisão do código.

O uso da ferramenta ocorreu principalmente nas seguintes partes:

- revisão da função `menos_buracos`, onde faltava a restrição que liga a variável de buraco b às variáveis a, m e n, e a sua inclusão no objetivo;
- discussão sobre a independência do validador e da contagem de buracos em relação ao modelo, levando a que `verificar`, `contar_buracos` e `custo_preferencias` passassem a reler os ficheiros CSV a partir do nome da pasta;

- uniformização do formato do horário como lista de tuplos (t, d, i, h), com a função `extrair_horario`, e adaptação das restantes funções a esse formato;
- definição da validação cruzada entre o valor do objetivo devolvido pelo solver e o cálculo independente dos buracos e das preferências;
- adaptação da construção incremental ao novo formato do horário e criação da função `guardar_dados` para validar os cenários alterados a partir de ficheiros;
- resolução de erros de execução no Marimo, como variáveis não definidas entre células, células com funções aninhadas e incompatibilidades com a versão instalada do OR-Tools (`StatusName`);
- fixação dos parâmetros do solver (`num_workers` e `random_seed`) para tornar os resultados reprodutíveis;
- diagnóstico do exemplo das preferências, que inicialmente não mostrava efeito porque a professora escolhida nunca tinha aulas nos tempos penalizados, e definição de um exemplo com efeito visível.

O código foi desenvolvido de forma incremental: era apresentada uma tentativa de implementação ou uma mensagem de erro, seguida da análise do problema e da respetiva correção. As sugestões foram testadas no notebook antes de serem mantidas, e os resultados apresentados no relatório foram obtidos pela execução do próprio notebook.

O diálogo utilizado durante a resolução encontra-se disponível em: https://claude.ai/share/af2f7552-ebd7-4024-ad27-f4447fdf0834